Tutorial 11 report
EE5311 (Digital IC design)

- Amogh G. Okade (EP22B020)

Question 1, 2

- Verilog code used for the 16-bit carry bypass adder (with a clock input added and the inputs/outputs of the adder registered) –

```
module cb16 (output reg[16:0] sum, input[15:0] a, input[15:0] b, input cin, input
clk);
    logic[15:0] a_q, b_q, p;
    logic cin_q;
    logic[16:0] sum_d;
    logic[3:0] cim;
    logic[3:0] com;

    genvar x;
    generate
        for (x = 0; x <= 15; x = x + 1) begin : pgen
            assign p[x] = a_q[x] ^ b_q[x];
        end
    endgenerate

    rc4 stag0 (sum_d[3:0], cim[0], a_q[3:0], b_q[3:0], cin_q);
    assign com[0] = (cim[0] & (p[3]&p[2]&p[1]&p[0])) | (cin_q &
~(p[3]&p[2]&p[1]&p[0]));
    rc4 stag1 (sum_d[7:4], cim[1], a_q[7:4], b_q[7:4], com[0]);
    assign com[1] = (cim[1] & (p[7]&p[6]&p[5]&p[4])) | (com[0] &
~(p[7]&p[6]&p[5]&p[4]));
    rc4 stag2 (sum_d[11:8], cim[2], a_q[11:8], b_q[11:8], com[1]);
    assign com[2] = (cim[2] & (p[11]&p[10]&p[9]&p[8])) | (com[1] &
~(p[11]&p[10]&p[9]&p[8]));
    rc4 stag3 (sum_d[15:12], cim[3], a_q[15:12], b_q[15:12], com[2]);
    assign com[3] = (cim[3] & (p[15]&p[14]&p[13]&p[12])) | (com[2] &
~(p[15]&p[14]&p[13]&p[12]));

    assign sum_d[16] = com[3];

    always @ (posedge clk) begin
        a_q <= a;
        b_q <= b;
        cin_q <= cin;
        sum <= sum_d;
    end
endmodule



module rc4 (output reg[3:0] sum, output cout, input[3:0] a, input[3:0] b, input
cin);
    logic[3:0] co;
    assign co[0] = (a[0] & b[0]) | (a[0] & cin) | (b[0] & cin);
    assign sum[0] = a[0] ^ b[0] ^ cin;
    genvar i;
    generate
       for (i=1; i<=3; i=i+1) begin : rcgen
              assign co[i] = (a[i] & b[i]) | (a[i] & co[i-1]) | (b[i] & co[i-1]);
              assign sum[i] = a[i] ^ b[i] ^ co[i-1];
       end
    endgenerate
    assign cout = co[3];
endmodule
```

Question 3

- The adder was synthesized using yosys without any errors –

```
2. Executing Verilog-2005 frontend: cb16.v
Parsing SystemVerilog input from `cb16.v' to AST representation.
Generating RTLIL representation for module `\cb16'.
Generating RTLIL representation for module `\rc4'.
Warning: reg '\sum' is assigned in a continuous assignment at cb16.v:39.12-39.38
.
Warning: reg '\sum' is assigned in a continuous assignment at cb16.v:44.14-44.44
.
Warning: reg '\sum' is assigned in a continuous assignment at cb16.v:44.14-44.44
.
Warning: reg '\sum' is assigned in a continuous assignment at cb16.v:44.14-44.44
.
Successfully finished Verilog frontend.
```

```
4.26. Executing CHECK pass (checking for obvious problems).
Checking module cb16...
Checking module rc4...
Found and reported 0 problems.
```

```
7.2. Executing DEMUXMAP pass.
Dumping module `\cb16'.
Dumping module `\rc4'.
```

- Unfortunately, since my Verilog implementation includes 2 modules to realize the 16-bit carry bypass adder (a 4-bit ripple carry adder used for each of the stages; and the adder's module itself), the command *show* does not result in a visualization of the netlist.

```
yosys> show

8. Generating Graphviz representation of design.
ERROR: For formats different than 'ps' or 'dot' only one module must be selected
```

Question 4

- On performing the STA, the following was the critical path –

```
% create_clock -name clk -period 3 {clk}
% report_checks -path_delay max
Startpoint: _122_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _170_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

  Delay    Time   Description
---------------------------------------------------------
   0.00    0.00   clock clk (rise edge)
   0.00    0.00   clock network delay (ideal)
   0.00    0.00 ^ _122_/CLK (sky130_fd_sc_hd__dfxtp_1)
   0.31    0.31 v _122_/Q (sky130_fd_sc_hd__dfxtp_1)
   0.38    0.69 v stag0/_52_/X (sky130_fd_sc_hd__maj3_1)
   0.26    0.95 ^ stag0/_62_/Y (sky130_fd_sc_hd__o21ai_0)
   0.14    1.09 v stag0/_64_/Y (sky130_fd_sc_hd__a21oi_1)
   0.37    1.47 v stag0/_65_/X (sky130_fd_sc_hd__maj3_1)
   0.18    1.65 v _099_/X (sky130_fd_sc_hd__a21o_1)
   0.38    2.03 v stag1/_52_/X (sky130_fd_sc_hd__maj3_1)
   0.26    2.29 ^ stag1/_62_/Y (sky130_fd_sc_hd__o21ai_0)
   0.14    2.43 v stag1/_64_/Y (sky130_fd_sc_hd__a21oi_1)
   0.35    2.79 v stag1/_65_/X (sky130_fd_sc_hd__maj3_1)
   0.14    2.93 ^ _106_/X (sky130_fd_sc_hd__lpflow_isobufsrc_1)
   0.08    3.00 v _107_/Y (sky130_fd_sc_hd__nor2_1)
   0.39    3.40 v stag2/_52_/X (sky130_fd_sc_hd__maj3_1)
   0.26    3.65 ^ stag2/_62_/Y (sky130_fd_sc_hd__o21ai_0)
   0.14    3.79 v stag2/_64_/Y (sky130_fd_sc_hd__a21oi_1)
   0.35    4.15 v stag2/_65_/X (sky130_fd_sc_hd__maj3_1)
   0.06    4.21 ^ _113_/Y (sky130_fd_sc_hd__nand2_1)
   0.09    4.30 v _114_/Y (sky130_fd_sc_hd__o31ai_1)
   0.40    4.70 v stag3/_52_/X (sky130_fd_sc_hd__maj3_1)
```

```
   0.26    4.96 ^ stag3/_62_/Y (sky130_fd_sc_hd__o21ai_0)
   0.14    5.10 v stag3/_64_/Y (sky130_fd_sc_hd__a21oi_1)
   0.35    5.44 v stag3/_65_/X (sky130_fd_sc_hd__maj3_1)
   0.27    5.71 v _120_/X (sky130_fd_sc_hd__mux2_1)
   0.00    5.71 v _170_/D (sky130_fd_sc_hd__dfxtp_1)
           5.71   data arrival time

   3.00    3.00   clock clk (rise edge)
   0.00    3.00   clock network delay (ideal)
   0.00    3.00   clock reconvergence pessimism
           3.00 ^ _170_/CLK (sky130_fd_sc_hd__dfxtp_1)
  -0.12    2.88   library setup time
           2.88   data required time
-----------------------------------------------------------
           2.88   data required time
          -5.71   data arrival time
-----------------------------------------------------------
          -2.84   slack (VIOLATED)
```

Question 5

- No, this is not the correct critical path. We see that the critical path used goes through the ripple carry adders in stages 1 and 2 also, which we know for a fact is not true. Also, STA claims that the last computed output is Cout, whereas it actually is S15.
- After removing these false paths and running the STA again –

```
% set_false_path -through cim[1]
% set_false_path -through cim[2]
% set_false_path -through com[3]
% report_checks -path_delay max
Startpoint: _122_ (rising edge-triggered flip-flop clocked by clk)
Endpoint: _169_ (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

  Delay    Time   Description
---------------------------------------------------------
   0.00    0.00   clock clk (rise edge)
   0.00    0.00   clock network delay (ideal)
   0.00    0.00 ^ _122_/CLK (sky130_fd_sc_hd__dfxtp_1)
   0.31    0.31 v _122_/Q (sky130_fd_sc_hd__dfxtp_1)
   0.38    0.69 v stag0/_52_/X (sky130_fd_sc_hd__maj3_1)
   0.26    0.95 ^ stag0/_62_/Y (sky130_fd_sc_hd__o21ai_0)
   0.14    1.09 v stag0/_64_/Y (sky130_fd_sc_hd__a21oi_1)
   0.37    1.47 v stag0/_65_/X (sky130_fd_sc_hd__maj3_1)
   0.25    1.71 ^ _105_/Y (sky130_fd_sc_hd__a211oi_1)
   0.16    1.88 v _114_/Y (sky130_fd_sc_hd__o31ai_1)
   0.40    2.27 v stag3/_52_/X (sky130_fd_sc_hd__maj3_1)
   0.26    2.53 ^ stag3/_62_/Y (sky130_fd_sc_hd__o21ai_0)
   0.14    2.67 v stag3/_64_/Y (sky130_fd_sc_hd__a21oi_1)
   0.13    2.81 v stag3/_67_/Y (sky130_fd_sc_hd__xnor2_1)
   0.00    2.81 v _169_/D (sky130_fd_sc_hd__dfxtp_1)
           2.81   data arrival time

   3.00    3.00   clock clk (rise edge)
   0.00    3.00   clock network delay (ideal)
   0.00    3.00   clock reconvergence pessimism
           3.00 ^ _169_/CLK (sky130_fd_sc_hd__dfxtp_1)
  -0.12    2.88   library setup time
           2.88   data required time
-----------------------------------------------------------
           2.88   data required time
          -2.81   data arrival time
-----------------------------------------------------------
           0.07   slack (MET)
```

- Here, we see that the critical path used is the correct critical path (consistent with the previous tutorial).

We notice that the new critical path ends at S15 (_169_), instead of Cout (_170_) and that it does not go through the 2nd and 3rd ripple carry adders.

- Screenshot from the *_synth.v file to substantiate the above point –

```
sky130_fd_sc_hd__dfxtp_1 _169_ (
  .CLK(clk),
  .D(sum_d[15]),
  .Q(sum[15])
);
sky130_fd_sc_hd__dfxtp_1 _170_ (
  .CLK(clk),
  .D(com[3]),
  .Q(sum[16])
);
```

Conclusion

We can see that the slack has improved significantly after the false paths were removed and can conclude that the maximum time period to run the clock at, is approximately 3ns (since a clock period of 2.9 results in a negative slack, even after removing the false paths).